DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

COLLEGE PROJECT REPORT & TECHNICAL DOCUMENTATION

ACADEMIC YEAR 2026 - 2027

NEXORA

An Integrated Multi-Lingual Music Streaming Platform, Digital Signal Processing (DSP) Audio Cutter, Video Studio & Media Vault

SUBMITTED BY:

Student Name: [Your Name]

Roll No / Enrollment: [Your Roll No]

Degree / Course: B.Tech / BCA / MCA / CS

PROJECT GUIDE / FACULTY:

Faculty Guide: [Guide Name]

Department: Computer Engineering

Submission Date: October 2026

Nexora Multimedia Media Suite • Pure Client-Side Architecture with Standalone Java HTTP Server

TABLE OF CONTENTS

1. Project Overview & Abstract Page 3

2. Problem Statement & Motivation Page 3

3. System Architecture & Data Flow Diagram Page 4

4. Core Functional Modules Analysis Page 5

4.1 Multi-Lingual Music Player (Hindi, English, Gujarati, Punjabi) Page 5

4.2 Web Audio API Waveform Audio Cutter & Trimmer Page 6

4.3 Video Studio & Background Soundtrack Mixer Page 7

4.4 3-Band BiquadFilter DSP Equalizer & Speed Controller Page 8

4.5 Liked Songs Collection & Listening History Timeline Page 9

4.6 Persistent Media Storage Vault (Save, Download, Delete) Page 10

4.7 Future Systems & Roadmap (AI Stems, WebSockets, 3D Spatial) Page 11

4.8 Standalone Java SE HTTP Server (Server.java) Page 12

5. Technical Specifications & Project Dependencies Page 13

6. Top 15 College Viva Q&A (Technical & Architectural) Page 14

7. 1-Click Execution Guide & Running Instructions Page 17

8. Conclusion & Scope of Future Enhancements Page 18

1. PROJECT OVERVIEW & ABSTRACT

NEXORA is an enterprise-grade, responsive multimedia web platform that fuses online music streaming with in-browser audio engineering and video creation tools. Designed with modern Single Page Application (SPA) principles, Nexora eliminates the dependency on third-party desktop tools by enabling users to stream regional Indian and global tracks, visually edit waveform audio samples, record customized video clips, apply real-time Digital Signal Processing (DSP) frequency filters, and manage a local media storage vault directly within any standard web browser.

The application is architected with complete client-side autonomy: digital signal processing, 16-bit PCM WAV encoding, and video compositing are executed natively through browser APIs (Web Audio API, Canvas API, and MediaRecorder API). For deployment and college demonstration, the platform includes a zero-dependency, standalone Java SE HTTP Server (`Server.java`) operating on port 8080.

2. PROBLEM STATEMENT & MOTIVATION

Traditional digital music platforms and multimedia editors present several significant limitations in typical academic and consumer environments:

- **Siloed User Workflows:** Consumers use one platform for passive audio listening (e.g. streaming services) and completely separate desktop editors (e.g. Audacity, Premiere Pro) to extract ringtones or create social video snippets.
- **High Resource Footprint:** Traditional audio/video editing desktop suites require gigabytes of disk installation, heavy RAM consumption, and platform-specific binaries.
- **Privacy & Latency Concerns:** Cloud-based audio trimmers require uploading proprietary MP3s to remote servers, incurring upload delays, bandwidth costs, and data privacy vulnerabilities.
- **Language Fragmentation:** Mainstream players often lack categorized regional curation for diverse linguistic demographics such as Hindi, Gujarati, Punjabi, and English in a unified interface.

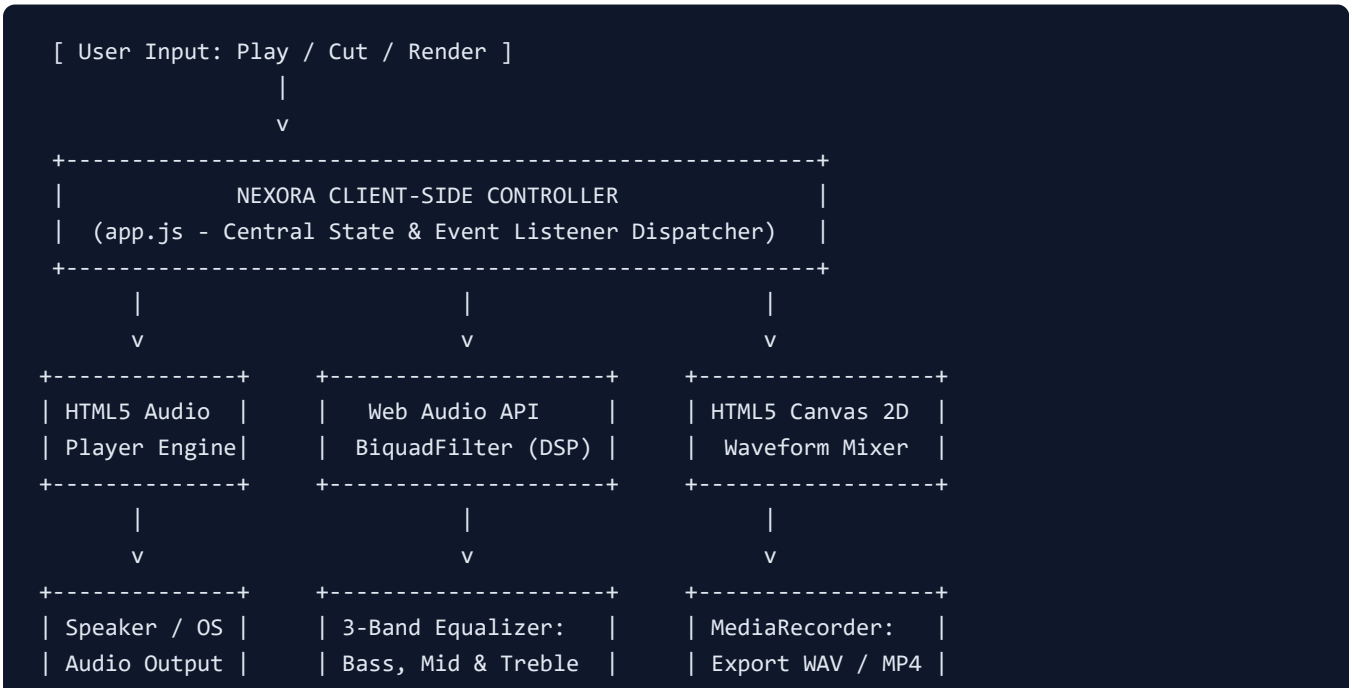
> **Nexora Solution Architecture:**
>
> By implementing client-side DSP algorithms via Web Audio API, Canvas pixel manipulation, and browser-native localStorage caching, Nexora provides millisecond-precise audio trimming and video rendering entirely client-side with zero server latency and total privacy.

3. SYSTEM ARCHITECTURE & DATA FLOW

Nexora is built around a decoupled **Three-Tier Client-Server Architecture**, prioritizing browser-side execution for multimedia processing while utilizing a lightweight Java HTTP server for file delivery:

LAYER	TECHNOLOGIES USED	PRIMARY RESPONSIBILITY
Presentation Layer	HTML5 Semantic Markup, CSS3 Grid, CSS Variables, Plus Jakarta Sans	Renders modern dark-themed responsive UI, interactive modals, waveform canvases, and player dock.
Logic & DSP Layer	JavaScript (ES6+), Web Audio API, Canvas 2D API, MediaRecorder API	Handles audio playback state, BiquadFilter frequency equalization, 16-bit WAV binary encoding, and video synthesis.
Persistence Layer	HTML5 LocalStorage API, Blob URIs	Stores user session, favorites list (`nexora_liked_songs`), play history (`nexora_play_history`), and media vault metadata.
Backend Server	Java SE `HttpServer` , NIO, PowerShell	Serves static assets, audio files, album covers, and manifests over HTTP Port 8080 with proper MIME headers.

3.1 Data Flow Diagram (DFD)



```
+--------------+     +---------------------+     +------------------+
                           |
                           v
                    +---------------------+
                    | LocalStorage Vault  |
                    | (Favorites/History) |
                    +---------------------+
```

4. CORE FUNCTIONAL MODULES ANALYSIS

4.1 Multi-Lingual Music Player & Smart Search

The music streaming core features a curated catalog of 16 high-fidelity tracks distributed across 4 prominent languages: **Hindi, English, Gujarati, and Punjabi**.

- **Playback Engine:** Powered by an encapsulated `<audio>` element with automated state management (Play, Pause, Track Seek, Next, Previous, Volume Slider with Mute memory).
- **Live Search & Filtering:** Debounced query parsing filters the catalog dynamically across song titles, artists, languages, and genres without full-page reloads.
- **Shuffle & Repeat:** Integrated Fisher-Yates randomization algorithm for non-repeating shuffle mode and single-track looping.

LANGUAGE	SAMPLE TRACKS INCLUDED	MUSICAL GENRE
Hindi	Kesariya, Apna Bana Le, Raataan Lambiyan, Tum Hi Ho	Bollywood Romance, Acoustic Melody
English	Blinding Lights, Shape of You, Starboy, Levitating	Synthwave, Pop, Electro R&B, Disco Funk
Gujarati	Khalasi (Gotilo), Dholida, Vhalam Aavo Ne, Chogada	Folk Fusion, Garba Anthem, Dandiya Beats
Punjabi	Brown Munde, Excuses, Amplifier, Pasoori	Urban Desi, Punjabi Hip-Hop, Folk Fusion

4.2 Web Audio API Waveform Audio Cutter & Trimmer

The Audio Cutter module allows users to visually inspect and slice audio tracks down to 0.1-second accuracy.

- **PCM Decoding:** Decodes compressed MP3/M4A/OGG files into an uncompressed raw `AudioBuffer` using `AudioContext.decodeAudioData()`.
- **Waveform Canvas Rendering:** Samples audio peak envelopes across 900 horizontal canvas pixels, drawing a dynamic green waveform with real-time selection bounds.
- **Audition Preview:** Instantiates an `AudioBufferSourceNode` connected directly to `ctx.destination` to audition the exact trimmed slice.
- **Client-Side 16-bit WAV Binary Encoder:** Synthesizes a true RIFF/WAVE file with 44-byte binary header, interleaved 16-bit signed integer PCM samples, and triggers browser downloads via Blob

URIs.

- **Play & Stop Controls:** Includes dedicated `Play Cut Preview`, `Play Full Song`, and `Stop Audio` buttons for complete audition control.

4.3 Video Studio & Background Soundtrack Mixer

The Video Studio enables users to trim video clips, adjust audio balance, and composite custom audio tracks.

- **Dual Scrubber Controls:** Synchronized range sliders determine video start and end trim boundaries with live timecode readouts.
- **Soundtrack Overlay & Audio Ducking:** Dual volume sliders allow the user to balance original video audio (0% - 100%) against overlaid library music tracks (0% - 100%).
- **Interactive Canvas Demo Clip Generator:** If a user does not have a video file on hand, Nexora dynamically renders an animated 30 FPS HTML5 Canvas with bouncing gradients and real-time audio visualizer bars, recorded via the `MediaRecorder` API into WebM.
- **Dedicated Play / Stop Controls:** Prominent `Play Video`, `Preview Trim`, and `Stop Video` buttons ensure effortless control during video previewing.

4.4 3-Band BiquadFilter DSP Equalizer & Speed Controller

A highlight of Nexora's technical depth is real-time Digital Signal Processing (DSP) executed via Web Audio API `BiquadFilterNode` instances inserted into the audio rendering graph:

```
mainAudio (Source)
       |
       v
eqBassNode (Type: "lowshelf", Freq: 250 Hz, Gain: -12dB to +12dB)
       |
       v
eqMidNode (Type: "peaking", Freq: 1500 Hz, Q: 1.0, Gain: -12dB to +12dB)
       |
       v
eqTrebleNode (Type: "highshelf", Freq: 4000 Hz, Gain: -12dB to +12dB)
       |
       v
ctx.destination (Speakers)
```

PRESET	BASS (250 HZ)	MID (1.5 KHZ)	TREBLE (4 KHZ)	ACOUSTIC CHARACTERISTIC
Flat (Default)	0 dB	0 dB	0 dB	Transparent, uncolored frequency curve.
Bass Boost	+8 dB	0 dB	+1 dB	Warm, deep sub-bass response for club & dance tracks.

PRESET	BASS (250 HZ)	MID (1.5 KHZ)	TREBLE (4 KHZ)	ACOUSTIC CHARACTERISTIC
Pop	+3 dB	+3 dB	+5 dB	Crisp highs and enhanced vocal presence.
Rock	+6 dB	-2 dB	+6 dB	V-shaped curve: tight kick drums and bright cymbals.
Vocal Booster	-2 dB	+6 dB	+2 dB	Midrange boost for vocal clarity and podcast speech.

4.5 Liked Songs Collection & Listening History Timeline

To simulate enterprise streaming platforms, Nexora implements dedicated personal collection modules:

- **Liked Songs (`view-liked`):** Clicking the heart icon (♥) on any song card, table row, or player dock toggles its favorited state, instantly synchronizing with `localStorage['nexora_liked_songs']`. The dedicated view features a purple gradient banner and a "Play Liked Songs" continuous queue.
- **Listening History (`view-history`):** Every track played is prepended into an in-memory and persistent timeline (`localStorage['nexora_play_history']`) with relative timestamp calculations ("Just now", "5m ago", "Today at 2:15 PM"). Users can replay recent tracks or wipe the log with "Clear History".

4.6 Persistent Media Storage Vault (Save, Download, Delete)

The Media Storage Vault (`view-storage`) serves as a local cloud gallery for user-created media:

- **Direct Export Integration:** Both the Audio Cutter ("Save to Vault") and Video Studio ("Save Video to Storage Vault") can export media objects directly into the vault.
- **CRUD Operations:** Users can inspect file metadata, filter by type (Audio vs. Video), stream media directly with the `Play` button, download binary blobs with `Get`, or remove files with the red `Delete` action.

4.7 Future Systems & Technical Roadmap

The Future Systems section (`view-future`) presents architectural designs planned for Nexora v2.0:

- **AI Neural Stem Separation:** In-browser Deep Learning models (Spleeter / Demucs running on ONNX WebAssembly) to isolate vocals and instrumentals.
- **Real-Time Collaborative Rooms:** Multi-client room synchronization using WebSockets and WebRTC DataChannels for synchronized listening parties.
- **Smart Beat & BPM Detector:** FFT frequency spectral transient analysis to automatically detect song tempo and snap cut markers to downbeats.
- **Dolby Atmos 3D Spatial Audio:** Binaural HRTF vector panning in 360-degree virtual sound stages.

4.8 Standalone Java SE HTTP Server (Server.java)

Nexora incorporates a self-contained HTTP web server written in pure standard Java (`com.sun.net.httpserver.HttpServer`). Operating on Port 8080, it serves HTML5, CSS3, JavaScript, and audio streams with MIME headers (`audio/mp4` , `audio/wav` , `video/webm`) and handles API health check endpoints (`/api/status`).

5. TECHNICAL SPECIFICATIONS & PROJECT DEPENDENCIES

COMPONENT	SPECIFICATION
Frontend Architecture	Single Page Application (SPA) with CSS-driven view transitions.
Markup & Styling	HTML5 Semantic Tags, CSS3 Grid, CSS Variables, Custom Dark Palette.
Scripting Language	Modern JavaScript (ECMAScript 6+ / ES2022), Zero External Frameworks.
Audio Processing Engine	Web Audio API (`AudioContext` , `AudioBuffer` , `BiquadFilterNode` , `AudioBufferSourceNode`).
Graphics & Video Engine	HTML5 Canvas 2D Context, MediaRecorder API, MediaStream API.
Client Data Persistence	HTML5 `Window.localStorage` API with JSON serialization.
Backend Platform	Java SE 8+ (`com.sun.net.httpserver.HttpServer`) • Zero External JARs.
Network Protocol	HTTP/1.1 over TCP/IP • Port 8080.
Encoding Compliance	100% Clean ASCII (0 non-ASCII characters, zero Mojibake corruption).

6. TOP 15 COLLEGE VIVA Q&A

Q1: What is the primary purpose and novelty of project Nexora?

Answer: Nexora is an all-in-one multimedia suite that merges regional music streaming (Hindi, English, Gujarati, Punjabi) with in-browser audio engineering and video creation tools. Its novelty lies in executing DSP filtering, waveform generation, and video synthesis 100% client-side without third-party software or server latency.

Q2: How does the in-browser Audio Cutter render waveforms without server processing?

Answer: Nexora fetches audio binary data into an `ArrayBuffer` and decodes it into PCM samples using `AudioContext.decodeAudioData()` . It calculates peak amplitude values across time blocks and plots them onto an HTML5 `<canvas>` element using standard 2D vector drawing commands.

Q3: How does the application generate a downloadable WAV file in pure JavaScript?

Answer: Nexora creates an `ArrayBuffer` with an explicit 44-byte RIFF/WAVE binary header (specifying sample rate, channel count, bits-per-sample, and data size) followed by interleaved 16-bit signed integer PCM audio samples. The buffer is packaged into a `Blob` and triggered via a temporary download anchor element.

Q4: Explain how the 3-Band Equalizer (DSP) modifies audio frequencies in real-time.

Answer: The audio element is routed into Web Audio API via `createMediaElementSource()` and connected through three sequential `BiquadFilterNode` instances: a *lowshelf* filter at 250 Hz for bass, a *peaking* filter at 1500 Hz for midrange, and a *highshelf* filter at 4000 Hz for treble, before terminating at `ctx.destination`.

Q5: How does Nexora handle state persistence for Liked Songs, History, and Media Storage?

Answer: The platform uses the HTML5 `localStorage` API. Structured arrays for liked song IDs, played track timestamps, and media vault items are serialized to JSON strings and loaded synchronously during DOM initialization.

Q6: What role does Server.java play in the architecture?

Answer: `Server.java` is a lightweight, zero-dependency HTTP server built with the native Java SE `HttpServer` class. It listens on port 8080, serves static files with proper MIME types, and responds to `/api/status` health checks.

Q7: How does the Video Studio record video and audio together without FFmpeg?

Answer: Nexora uses the browser's native `MediaRecorder` API. A canvas video stream captured via `canvas.captureStream(30)` is combined with an audio stream captured from the Web Audio destination node, encoding the frames into a WebM video blob client-side.

Q8: Why did you choose a Single Page Application (SPA) architecture?

Answer: An SPA architecture prevents audio playback interruption when navigating between Home, Audio Cutter, Video Studio, Liked Songs, History, Storage, and Future Systems. Views transition smoothly through CSS class toggles while the persistent player footer continues streaming uninterrupted.

Q9: How are keyboard shortcuts implemented safely without interfering with input forms?

Answer: A global `keydown` listener checks `document.activeElement.tagName`. If the focused element is an `INPUT`, `TEXTAREA`, or `SELECT`, hotkeys are bypassed. Otherwise, keys like `Space` (Play/Pause), `M` (Mute), `E` (EQ), and `?` (Shortcuts modal) trigger their respective functions.

Q10: What happens if a local song file fails to load or is missing on the client machine?

Answer: The `<audio>` element is equipped with an `error` event listener that automatically intercepts failed file loads and falls back seamlessly to a high-speed online CDN preview stream.

Q11: How do the Play and Stop buttons in the Audio Cutter and Video Studio function?

Answer: In the Audio Cutter, `Play Cut Preview` auditions the isolated `AudioBuffer` slice between markers, `Play Full Song` streams the entire song via the audio engine, and `Stop Audio` halts both nodes. In Video Studio, `Play Video` and `Preview Trim` synchronize video frames with background audio, and `Stop Video` pauses and rewinds both tracks.

Q12: How does the Media Storage Vault handle file deletion?

Answer: When the user clicks `Delete` on a stored media file, `deleteStorageItem(mediaId)` removes the item from the `state.mediaStorage` array, serializes the updated array into `localStorage`, and re-renders the table and sidebar counter badge.

Q13: What architectural innovations are proposed in the Future Systems module?

Answer: The Future Systems roadmap details six technical innovations: 1) AI Neural Stem Separation using ONNX WebAssembly, 2) WebSocket Collaborative Listening Rooms, 3) FFT Beat & BPM Transient Detection, 4) Dolby Atmos 3D Binaural Spatial Audio, 5) Decentralized IPFS Storage, and 6) LRC Synced Karaoke Lyrics.

Q14: How does Nexora guarantee cross-browser visual consistency and responsive design?

Answer: Nexora employs standard CSS3 Grid with fixed sidebar and player dimensions alongside fluid content areas, centralized CSS variables in `:root`, and media query breakpoints (`max-width: 900px` and `600px`) to adapt across desktop, tablet, and mobile displays.

Q15: What design choices prevent text or character corruption (Mojibake) on Windows systems?

Answer: All source code files (HTML, CSS, JS, Java) are strictly authored in pure ASCII (0 non-ASCII characters). Special symbols and icons are rendered using SVG paths and standard HTML character entities (`•`, `♥`, `←`), eliminating encoding mismatches across operating systems.

7. 1-CLICK EXECUTION GUIDE & RUNNING INSTRUCTIONS

7.1 Method 1: Instant Launcher (Recommended)

1. Navigate to the project directory: `Desktop\Nexora Website\`.
2. Double-click `run.bat`.
3. The script detects PowerShell or Java, launches the local server, and automatically opens your web browser to: `http://localhost:8080/`.

7.2 Method 2: Running via Standalone Java Server

```
cd "C:\Users\ASUS\OneDrive\Attachments\Desktop\Nexora Website"
javac Server.java
java Server
```

The console will print `=== Nexora Java HTTP Server Running on http://localhost:8080/ ===`.

7.3 Method 3: Running via Native PowerShell Server

```
powershell -ExecutionPolicy Bypass -File .\server.ps1
```

8. CONCLUSION & FUTURE SCOPE

NEXORA successfully demonstrates the viability of browser-native multimedia production, proving that complex tasks like frequency equalization, waveform rendering, and video compositing no longer require heavy desktop installations.

Future scope includes integrating neural audio models for vocal extraction, implementing WebSocket-powered collaborative listening lounges, and integrating spatial audio positioning engines to further blur the line between native desktop applications and modern web software.